Populr — Engineering Onboarding

Internal engineering handbook. Read before your first pull request.

What Populr is

An AI CMO for early-stage companies. A founder pastes their website; Populr reads it, plans marketing campaigns, writes the content, publishes it across platforms, measures what happened and folds the result back into what it does next.

The product is not a chat wrapper. Most of the intelligence is deterministic engines with an LLM used at specific, bounded points.

Stack

- **Next.js 16** (App Router), React, TypeScript strict
- **Neon** serverless Postgres
- **Plain CSS** — one stylesheet, `app/globals.css`. No Tailwind, no CSS-in-JS
- **Vitest** for tests
- **Vercel** for hosting and cron

Dynamic routes use `ctx: { params: Promise<{ id: string }> }` — Next 16 made params async.

Running it

```
npm install
npm run dev          # http://localhost:3000
npx vitest run       # full suite
npx tsc --noEmit     # typecheck
npm run build        # production build
```

Without a database or API keys the product still runs: every store has an in-memory implementation and every AI path has a deterministic fallback. This is deliberate — see "Graceful degradation" below.

Environment: `GROQ_API_KEY`, `GEMINI_API_KEY` or `OPENAI_API_KEY` for generation; `DATABASE_URL` for Neon; `CRON_SECRET` for scheduled jobs.

The shape of the codebase

```
app/
  page.tsx            landing
  app/                the signed-in dashboard
  studio/             creative studio (content, launch, publishing, market, learning)
  api/                route handlers, one folder per resource
lib/
```

```
  services/llm.ts     the ONLY place that calls a model provider
  content/            composer, generation context, AI wiring
  ugc/                UGC scripts, hooks, versions
  launch/             launch planner + workspace state
  execution/          campaign execution engine (M14)
  agents/             the seven AI agents (M15)
  social/             cross-platform publishing (M12)
  market/             market intelligence (M13)
  learning/           pattern library, brand DNA (M10)
  jobs/               background job engine (M11)
  automation/         scheduled/recurring publishing
db/migrations/        plain SQL, applied in filename order
tests/                one file per subsystem
```

Six patterns you need to know

1. Repository pattern, everywhere

Every store has two implementations behind one interface:

```
export interface ThingRepo { get(...): Promise<T>; save(t: T): Promise<T>; }
export class InMemoryThingRepo implements ThingRepo { ... }
export class NeonThingRepo implements ThingRepo { ... }
```

A `shared.ts` picks one based on whether `db()` returns a connection. Tests use in-memory and never touch a database. If you add a store, follow this or your code becomes untestable.

2. Adapters own provider specifics

Platform rules (character limits, whether media is required, whether video is allowed) live in `lib/social/adapters.ts` and are read through `createAdapterRegistry()`. Nothing outside that file hardcodes "280 characters".

The same pattern applies to LLM providers (`lib/services/llm.ts`) and market data sources (`lib/market/sources.ts`). Core services only see normalized types.

3. One orchestrator per concern

- **Job Engine** (M11) owns background work, queueing and retries.
- **Publishing Engine** (M12) owns publishing, backoff and the dead-letter queue.
- **Execution Engine** (M14) owns campaign workflow state.
- **LLM service** owns provider routing, fallback and caching.

If you need one of those behaviours, call the engine. Do not build a second one. Several subsystems were deliberately written as thin layers over these — `lib/automation/runner.ts` publishes nothing itself; it claims a slot and hands it to M12.

4. Deterministic by default, LLM at the edges

Planning, scheduling, health scoring, recurrence and command parsing are all deterministic functions with injectable clocks. That is what makes them testable and what makes an approved plan the plan

that ships.

The LLM is used for writing content and a few reasoning summaries. Every one of those paths has a deterministic fallback that is used when no provider is configured or a response cannot be parsed — and the result is **marked** as such (`source: "deterministic"`) rather than passed off as model output.

5. Graceful degradation is a requirement, not a nicety

A dead market source degrades the market section, not the request. A missing database degrades to in-memory. A failing provider degrades to the deterministic path. In every case the response says what was missing.

The rule: **never silently produce a worse result.** If quality dropped, the payload says so and the UI shows it.

6. Idempotency on anything that touches the outside world

Publishing carries an idempotency key the Publishing Engine de-duplicates on. Automation slot ids are content-addressed hashes of (automation, time), so re-expanding a schedule is a no-op. State transitions are guarded — a slot moves `upcoming` → `publishing` through a check that fails if it isn't currently upcoming, so two cron runs cannot both claim it.

The milestones, briefly

	What it owns
M10 Learning Engine	Pattern library, Brand DNA, decision feedback
M11 Job Engine	Background jobs, queue, worker pool, SSE progress
M12 Cross-Platform Publishing	OAuth, encrypted tokens, 6 platform adapters, scheduler, retries
M13 Market Intelligence	Trends, competitors, keywords, opportunities, Market Memory
M14 Execution Engine	Campaign workflow state machine, health, notifications, adaptive timeline
M15 AI Team	Seven agents that perform the workflow steps

Each has a doc in `docs/milestone-*.md`. Read the one for the area you're touching.

The AI team (M15) — the rule that matters

Seven agents: Research, Strategy, Content, Creative, Publishing, Analytics, Learning.

Agents never orchestrate. They cannot call each other and cannot start work. The Execution Engine hands one agent one step; the agent's only inputs are the assembled context and that step, and its only output is a task record. What one agent learns reaches the next through the Business Graph, Market Memory and the Learning Engine.

This is enforced structurally — there is no handle to another agent to call.

Testing

`npx vitest run` — currently 546 tests across 53 files.

What we test is the **contract**, not the implementation:

- Illegal state transitions are refused (a published post cannot be un-published)
- A paused agent leaves no task record, so resuming replays no side effects
- Platform limits come from the adapters, asserted against the real constraint values
- Fallbacks mark themselves and reduce confidence
- The same inputs produce the same ids (idempotence)

Deterministic engines take an injectable `now()`. Never call `Date.now()` inside a pure function you want to test.

When you find a bug, write the test that catches it before you fix it. Several tests in the suite are named after the exact bug they prevent regressing.

Conventions

- **Comments explain why, not what.** If a line is surprising, say why it is that way.
- **Errors are sentences.** A raw API code must never reach a user's screen; map codes to

language that says what happened and what to do.

- **No placeholder UI.** No "Coming soon" badges on working features, no dead buttons.
- **Secret guard before committing.** `git status --short` must never show `.env.local`,

`*_key.txt` or `node_modules`.

- Commits describe the decision and its reason, not just the change.

Database migrations

Plain SQL in `db/migrations/`, applied in filename order (`YYYYMMDD_name.sql`). `RUNTIME_DDL` gates request-time table creation for local development; production applies migrations explicitly. Adding a table means adding a migration **and** the `CREATE TABLE IF NOT EXISTS` in the Neon store, so local dev works without a migration step.

Where to start

1 Run the app, paste a website, watch the dashboard populate.

2. Read `lib/services/llm.ts` — it is the spine of every AI path.
3. Read `lib/social/engine.ts` and one adapter — the clearest example of the patterns.
4. Read `tests/campaign-execution.test.ts` — it shows what we consider worth asserting.
5. Pick a `docs/milestone-*.md` for the area you'll work in.

What we care about in review

Does it reuse the engine that already does this? Does it degrade honestly? Is it testable without a network? Does the error text help someone who isn't you? Can it double-post?

If the answer to the last one is "probably not", that isn't good enough — make it impossible, and write the test that proves it.